

STAR Robot Control System

Comprehensive Technical Documentation

Complete Reference for Software Engineers

Version 1.0
December 2025

STAR Development Team
Locked Inc.

Contents

1	Nanopb (Protobuf) Protocol with ARQ and CRC for SPI Communication	2
1.1	Executive Summary	2
1.2	Key Design Decisions	2
1.2.1	Protocol Stack	2
1.2.2	Memory Budget	3
1.3	ARQ Communication Flow	3
1.3.1	Normal Operation (No Retransmission)	3
1.3.2	Error Recovery Scenarios	4
1.4	Error Handling Strategies	5

1 Nanopb (Protobuf) Protocol with ARQ and CRC for SPI Communication

1.1 Executive Summary

Overview: Protocol Buffers (nanopb) encapsulated in frames with CRC-32 and Automatic Repeat Request (ARQ) for reliable SPI communication between RPi5 and ESP32-S3.

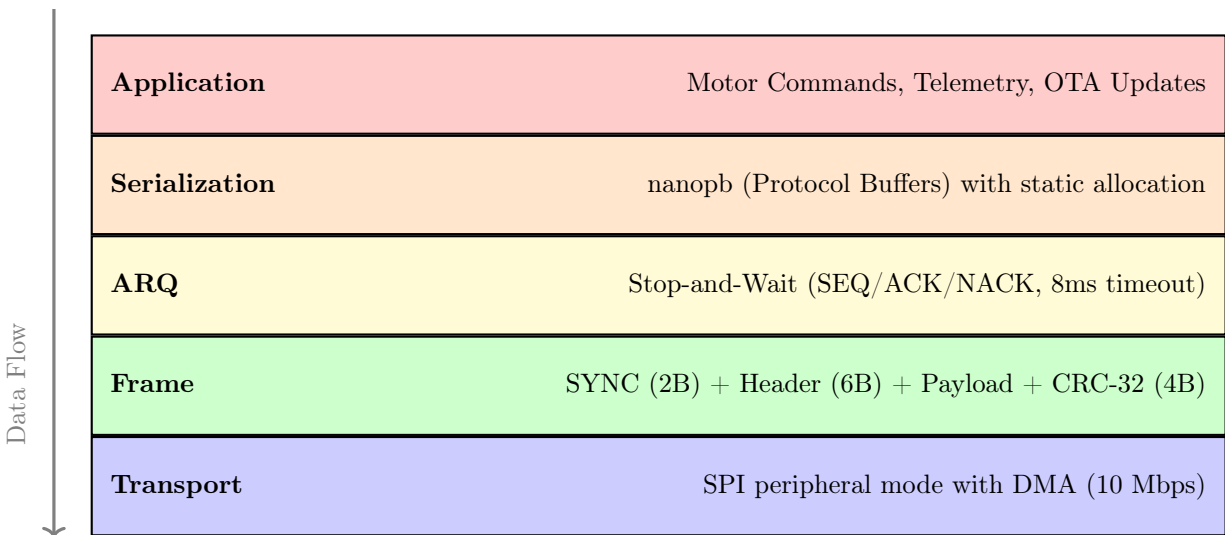


System Constraints:

- ESP32-S3-WROOM-1-N16: 16MB flash, 512KB SRAM, NO PSRAM
- 100Hz SPI communication (10ms period)
- 250Hz motor control loop (4ms period)
- Static allocation only (no malloc)

1.2 Key Design Decisions

1.2.1 Protocol Stack



1.2.2 Memory Budget

Table 1: SRAM Usage (512 KB total)

Component	Size (KB)	Usage (%)
DMA double-buffer	8.0	1.56
RX/TX buffers	2.0	0.39
Protobuf structs	1.5	0.29
ARQ state	4.5	0.88
Total	16.0	3.12

Table 2: Flash Usage (16 MB total)

Component	Size (KB)	Usage (%)
Generated protobuf	40	0.24
Nanopb runtime	12	0.07
Frame + ARQ layers	8	0.05
Total	60	0.37

Memory Budget Calculations:

SRAM (512 KB total):

- **DMA double-buffer:** 2×4092 bytes = 8184 bytes = 8.0 KB
Calculation: Two ping-pong DMA buffers at `STAR_SPI_DMA_MAX_SIZE` (4092 bytes each)
Usage: $\frac{8.0}{512} \times 100 = 1.56\%$
- **RX/TX buffers:** $1024 + 1024 = 2048$ bytes = 2.0 KB
Calculation: 1 KB RX buffer + 1 KB TX buffer for staging protobuf messages
Usage: $\frac{2.0}{512} \times 100 = 0.39\%$
- **Protobuf structs:** 1536 bytes = 1.5 KB
Calculation: Largest message structures (SetVelocityResponse $\approx$ 600 bytes, TelemetryData $\approx$ 400 bytes)
Usage: $\frac{1.5}{512} \times 100 = 0.29\%$
- **ARQ state:** $4 + 4 + 8 + 4 + 4092 = 4112$ bytes $\approx$ 4.5 KB
Calculation: Sequence number (4B) + retry counter (4B) + timestamp (8B) + state (4B) + retry buffer (4092B)
Usage: $\frac{4.5}{512} \times 100 = 0.88\%$
- **Total SRAM:** $8.0 + 2.0 + 1.5 + 4.5 = 16.0$ KB
Total usage: $\frac{16.0}{512} \times 100 = 3.12\%$

Flash (16 MB = 16384 KB total):

- **Generated protobuf:** 40 KB
Calculation: 3 proto files (common.proto, motor_control.proto, telemetry.proto) $\times$ 13 KB avg compiled size
Usage: $\frac{40}{16384} \times 100 = 0.24\%$
- **Nanopb runtime:** 12 KB
Calculation: Compiled size of pb_encode.c, pb_decode.c, pb_common.c
Usage: $\frac{12}{16384} \times 100 = 0.07\%$
- **Frame + ARQ layers:** 8 KB
Calculation: Framing logic + CRC-32 + ARQ state machine + retransmission logic
Usage: $\frac{8}{16384} \times 100 = 0.05\%$
- **Total Flash:** $40 + 12 + 8 = 60$ KB
Total usage: $\frac{60}{16384} \times 100 = 0.37\%$

1.3 ARQ Communication Flow

1.3.1 Normal Operation (No Retransmission)

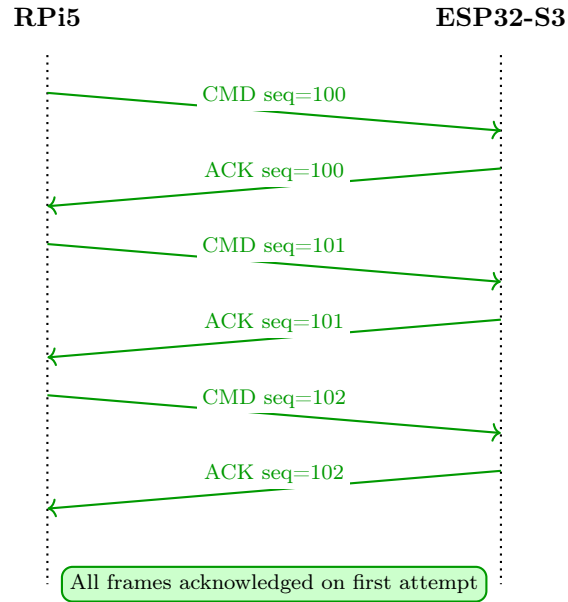


Figure 1: Normal ARQ Flow - Three Successful Transactions

1.3.2 Error Recovery Scenarios

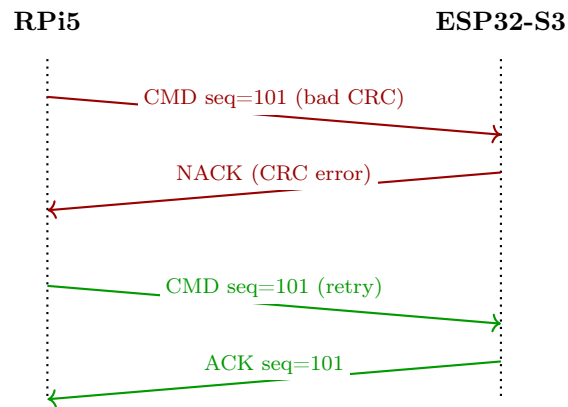


Figure 2: CRC Error - Retry with Same Sequence Number

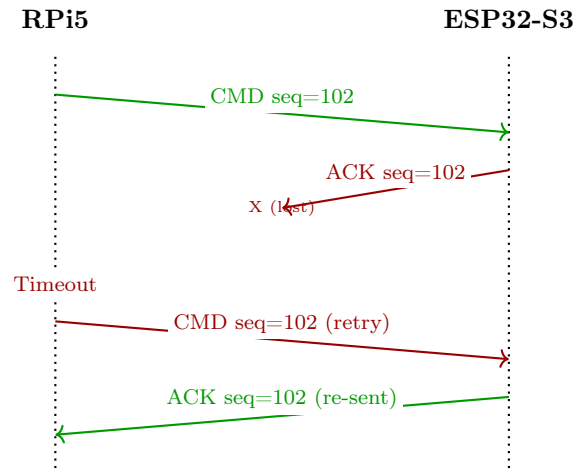


Figure 3: Duplicate Packet Scenario (Lost ACK)

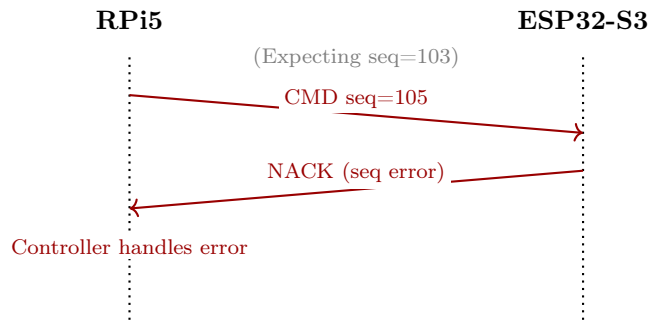


Figure 4: Out-of-Order Packet Scenario

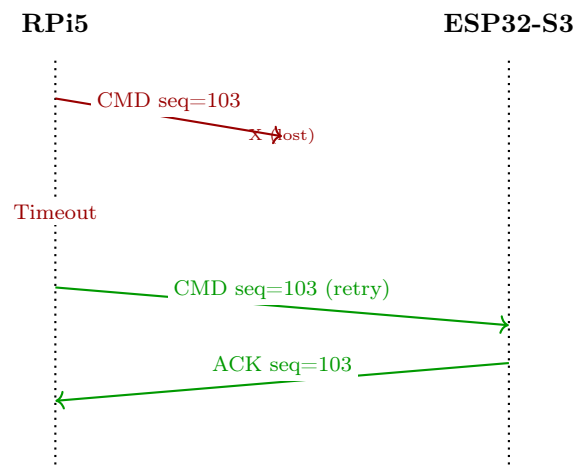


Figure 5: Lost Packet Scenario (Timeout)

1.4 Error Handling Strategies

CRC Mismatch

Action: NACK with CRC error reason
Recovery: Controller retry
Logging: Increment `crc_errors`

Sequence Error

Duplicate: Re-send ACK
Out-of-order: NACK + reject
Recovery: Controller retry

Timeout

RX timeout: Continue next cycle
500ms watchdog: Emergency stop
Retry fail: Log error

Invalid Protobuf

Action: NACK protocol error
Validate: IDs, ranges, offsets
Logging: Increment `proto_errors`